

A Scalable Distributed Search Architecture Using a Multi-Node Elasticsearch Cluster

Felix-An Pham Ponton
Gina Cody School of Engineering and
Computer Science, Concordia
University
Montreal, Quebec
f_phampo@live.concordia.ca

Refat Abuzriba
Gina Cody School of Engineering and
Computer Science, Concordia
University
Montreal, Quebec
r_abuzri@live.concordia.ca

Karim Moussa
Gina Cody School of Engineering and
Computer Science, Concordia
University
Montreal, Quebec
ka_mouss@live.concordia.ca

Philippe Lizotte
Gina Cody School of Engineering and
Computer Science, Concordia
University
Montreal, Quebec
p_lizot@live.concordia.ca

Sami Khalayli
Gina Cody School of Engineering and
Computer Science, Concordia
University
Montreal, Quebec
s_khalay@live.concordia.ca

ABSTRACT

This project presents a distributed search system built on a three-node Elasticsearch cluster deployed with Docker. A Python bulk-ingestion pipeline processes over 4.6 million Amazon Review documents, enabling parallel indexing and real-time retrieval. We evaluate key distributed properties—scalability, fault tolerance, and load balancing—through shard distribution analysis, ingestion performance, and simulated node failures. Results show that Elasticsearch’s sharded and replicated architecture provides efficient parallelism, resilient query execution, and continuous availability, demonstrating its effectiveness as a scalable platform for large-scale document analytics.

1 INTRODUCTION

Modern data-driven applications rely on distributed systems capable of storing, indexing and analyzing large-scale datasets with low latency and high availability. In this project, we used a three-node Elasticsearch cluster, complemented with Kibana for analytics and visualization, to ingest an Amazon Review Dataset that contains more than 170,000 products and 4,600,000 reviews of products. A typical setup that allowed us to exploit core principles of distributed systems such as scalability, fault tolerance and distributed computing.

Elasticsearch is a distributed, document-oriented NoSQL database designed for horizontal scalability and real-time search. It distributes both data and queries across multiple nodes. This parallel execution and the optimized indexing allow it to perform real-time full-text search with very low latency.

This report is organized in three main sections. First, we present the system architecture, detailing the design of our Elasticsearch cluster. Next, we examine key distributed features and concept behind Elasticsearch. Finally, we present our experimental tests, where we evaluate the cluster’s performance, scalability and reliability under real data workload.

2 SYSTEM ARCHITECTURE

Our system is built around a three-node Elasticsearch cluster deployed using Docker Compose. Each node participates in distributed indexing and search operations, providing scalability and fault tolerance through sharding and replication. A kibana instance is connected to the cluster to support data exploration, visualization and cluster monitoring. A python-based ingestion pipeline is used to load and index the Amazon Review data. while a Flask web application provide an interactive user interface for searching and browsing the indexed data. (Figure 1) A three-node deployment represents the smallest fully fault-tolerant configuration recommended for production use [2]. With three node, the system can maintain quorum during master elections and operating even if one node fails. This ensure high availability and consistent cluster-state management while allowing horizontal scaling of indexing and search workloads. In this architecture, data storage, ingestion and clients interfaces are cleanly separated. Kibana and the Flask application operate outside the cluster and communicate with through REST api. Avoiding unnecessary coupling between services and allow each component to scale, restart or update independently. Kibana serves as the visualization and monitoring interface, while the ingestion pipe functions exclusively as a data producer, leaving the storage, indexing and query execution responsibilities to the cluster.

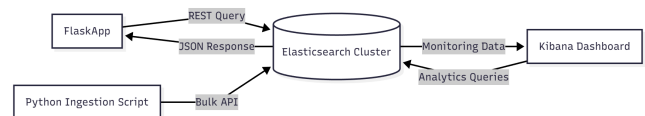


Figure 1: System architecture overview

2.1 Cluster Node Architecture

The cluster contains three homogeneous Elasticsearch nodes es01, es02 and es03 where each node is capable of performing multiple roles, master-eligible and data node. When elected, the master node is responsible for creating, updating and distributing the cluster state

such as master election, shard allocation, and metadata management such as index mappings and node membership. The elected master node publishes each updated cluster state to all other nodes, which must acknowledge and apply it before new operations continue. This ensures the cluster remains consistent.[1] At the same time, each node can function as a data, responsible for storing primary and replica shards, executing indexing operations and processing search queries. The shard configuration distributes documents to exploit parallelism. Our cluster uses three shard, matching the number of node in the system. It ensures that each node hold exactly one primary shard.

This multi-role configuration provides several advantages. First, redundancy ensures that the cluster remains functional even if a node fails. Second, parallel load distribution allows indexing and search workload to be spread across all three nodes, improving performance and throughput. Finally, load balancing is achieved by evenly distributing shards across the nodes, enhancing scalability and consistent performance.

2.2 Data Flow

The Amazon Review dataset consists of JSONL documents, each representing a product review and the amazon Products dataset consist of a products information such as ASIN, price, detail, description and more. the product and their corresponding reviews are linked through ASIN field. To ingest data into the cluster, we developed a Python ingestion script that processes files sequentially and performs the preprocessing needed to insert documents into the Elasticsearch index using the bulk API to stream documents to Elasticsearch. The ingestion process groups documents into batches and handles the request timeouts and automatically retries operations when necessary to maintain stability under load. Before documents are indexed, both datasets are preprocess to ensure consistent structure and high quality data by sanitizing missing or malformed fields. In particular, the "detail" field of the products dataset was originally store as a deeply nested json was normalized into a clean Python dictionary allowing the use of the flattened elasticSearch type. For the review dataset, we enrich it by generating lightweight semantic tags for each review textual content. This is performed using the NLTK library where we tokenize the text, remove punctuation and stop word, filter invalid token, and eliminate duplicate. the resulting tag list is stored alongside the review document and is used in a word cloud visualization by Kibana.

2.3 Flask Application Layer

The system includes a flask-based web application that serves as the primary user-facing component. the application provides a search interface that allows user to query the dataset based on product title. It dynamically construct Elasticsearch queries based on user input and processes the return results into formatted HTML page. The Flask application remains external to the Elasticsearch cluster. Communicates with nodes via the REST API. This separation simplifies maintenance and makes the application layer modular.

2.4 Docker Compose

The entire system is deployed using Docker, where each Elasticsearch node and the kibana interface run as independent containers.

This containerized setup isolates each service, while allowing them to communicate within a shared virtual network created by Docker Compose. Persistent data is stored using Docker volume, which maps the internal Elasticsearch data directory to host-managed storage so that index files and cluster metadata survive containers restart. The service orchestration is handled entirely by Docker compose, which defines the environment variables, networking configuration, and startup order of all components, ensuring consistent and reproducible deployment.

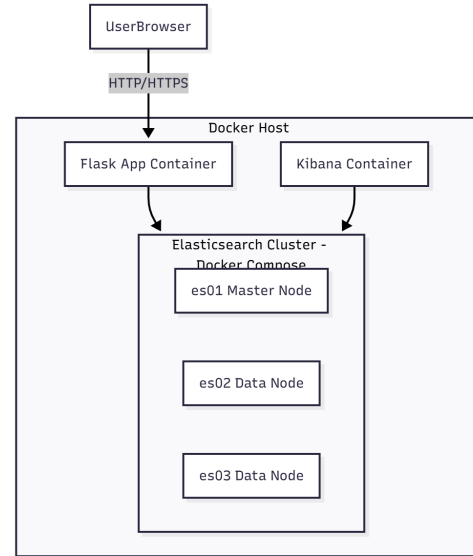


Figure 2: Complete Docker Setup

3 DISTRIBUTED FEATURES & CONCEPTS

Our system demonstrates three essential distributed system features: scalability, fault tolerance, and load balancing. These capabilities are enabled through a three-node Elasticsearch cluster deployed using Docker and configured with a deliberately sharded and replicated indexing strategy. Together, these mechanisms illustrate how distributed search systems achieve high performance, availability, and resilience.

3.1 Scalability

Scalability refers to the system's ability to accommodate increasing data volume or query load by adding resources rather than replacing existing components. In our design, scalability is achieved by distributing the workload across three Elasticsearch nodes (es01, es02, and es03). The dataset, which contains over 4.6 million documents, is indexed into an Elasticsearch index configured with three primary shards. Each shard is stored on a different node, enabling the ingestion and processing of data to occur in parallel rather than sequentially on a single machine. This distribution also applies to search operations, where queries are executed concurrently across all shards and aggregated by a coordinating node. Furthermore, the system can be expanded simply by adding additional nodes to the cluster, after which Elasticsearch automatically rebalances

shard placement. This demonstrates true horizontal scalability, a key characteristic of distributed architectures.

3.2 Fault Tolerance

Fault tolerance ensures that the system remains operational even when individual components fail. Our system provides fault tolerance through Elasticsearch's replication mechanism. Each of the three primary shards has a corresponding replica stored on a different node, ensuring that every piece of data exists in at least two independent locations. If a node containing a primary shard fails, Elasticsearch immediately promotes the corresponding replica on another node to primary status, allowing the system to continue processing queries without interruption. During such an event, the cluster temporarily enters a yellow health state to indicate reduced redundancy, but all functionality remains available. Once the failed node is restored, shards are automatically reallocated and redundancy is re-established, returning the cluster to a green state. This behavior demonstrates practical fault tolerance without data loss or manual recovery.

3.3 Load Balancing

Load balancing in distributed systems ensures that computational and storage load are evenly distributed across available resources. In our system, load balancing occurs naturally through Elasticsearch's sharding and replication mechanisms. During ingestion, each shard processes its assigned portion of the dataset independently, preventing bottlenecks that would occur if a single node handled the entire workload. For search operations, any node may act as the coordinating node, receiving the query and distributing it to the appropriate shards. Elasticsearch intelligently selects between primary and replica shards based on availability and load, which further distributes query processing across the cluster. Additionally, Docker's internal networking allows seamless communication among nodes and provides Kibana with a unified entry point into the system. As a result, both indexing and search workload are effectively balanced across the three nodes, improving performance and system stability.

4 EXPERIMENTS/TESTS

4.1 Test 1 — Node Failure



Figure 3: Cluster status before and after removing a node

This test validates the fault tolerance of our Elasticsearch cluster by simulating the failure of a single node. When a node goes offline, the cluster remains fully operational: Elasticsearch automatically promotes replica shards to primary status, ensuring uninterrupted

access to all data. During this period, the cluster enters a yellow state (see fig. 3, right), indicating reduced redundancy but preserved functionality.

Because primary and replica shards are distributed across multiple nodes, all documents remain searchable, and no data is lost. Once the failed node is restored, Elasticsearch automatically reallocates and re-balances shards, returning the cluster to a green state with full redundancy (see fig. 3, center).

4.2 Test 2 — Shard Distribution

Shard ID	Type	Size	Nodes
0	video-game-products	100 MB	node-0, node-1, node-2
1	video-game-products	100 MB	node-0, node-1, node-2
2	video-game-products	100 MB	node-0, node-1, node-2
3	video-game-products	100 MB	node-0, node-1, node-2
4	video-game-products	100 MB	node-0, node-1, node-2
5	video-game-products	100 MB	node-0, node-1, node-2
6	video-game-products	100 MB	node-0, node-1, node-2
7	video-game-products	100 MB	node-0, node-1, node-2
8	video-game-products	100 MB	node-0, node-1, node-2
9	video-game-products	100 MB	node-0, node-1, node-2
10	video-game-products	100 MB	node-0, node-1, node-2
11	video-game-products	100 MB	node-0, node-1, node-2
12	video-game-products	100 MB	node-0, node-1, node-2
13	video-game-products	100 MB	node-0, node-1, node-2
14	video-game-products	100 MB	node-0, node-1, node-2
15	video-game-products	100 MB	node-0, node-1, node-2
16	video-game-products	100 MB	node-0, node-1, node-2
17	video-game-products	100 MB	node-0, node-1, node-2
18	video-game-products	100 MB	node-0, node-1, node-2
19	video-game-products	100 MB	node-0, node-1, node-2
20	video-game-products	100 MB	node-0, node-1, node-2
21	video-game-products	100 MB	node-0, node-1, node-2
22	video-game-products	100 MB	node-0, node-1, node-2
23	video-game-products	100 MB	node-0, node-1, node-2
24	video-game-products	100 MB	node-0, node-1, node-2
25	video-game-products	100 MB	node-0, node-1, node-2
26	video-game-products	100 MB	node-0, node-1, node-2
27	video-game-products	100 MB	node-0, node-1, node-2
28	video-game-products	100 MB	node-0, node-1, node-2
29	video-game-products	100 MB	node-0, node-1, node-2
30	video-game-products	100 MB	node-0, node-1, node-2
31	video-game-products	100 MB	node-0, node-1, node-2
32	video-game-products	100 MB	node-0, node-1, node-2
33	video-game-products	100 MB	node-0, node-1, node-2
34	video-game-products	100 MB	node-0, node-1, node-2
35	video-game-products	100 MB	node-0, node-1, node-2
36	video-game-products	100 MB	node-0, node-1, node-2
37	video-game-products	100 MB	node-0, node-1, node-2
38	video-game-products	100 MB	node-0, node-1, node-2
39	video-game-products	100 MB	node-0, node-1, node-2
40	video-game-products	100 MB	node-0, node-1, node-2
41	video-game-products	100 MB	node-0, node-1, node-2
42	video-game-products	100 MB	node-0, node-1, node-2
43	video-game-products	100 MB	node-0, node-1, node-2
44	video-game-products	100 MB	node-0, node-1, node-2
45	video-game-products	100 MB	node-0, node-1, node-2
46	video-game-products	100 MB	node-0, node-1, node-2
47	video-game-products	100 MB	node-0, node-1, node-2
48	video-game-products	100 MB	node-0, node-1, node-2
49	video-game-products	100 MB	node-0, node-1, node-2
50	video-game-products	100 MB	node-0, node-1, node-2
51	video-game-products	100 MB	node-0, node-1, node-2
52	video-game-products	100 MB	node-0, node-1, node-2
53	video-game-products	100 MB	node-0, node-1, node-2
54	video-game-products	100 MB	node-0, node-1, node-2
55	video-game-products	100 MB	node-0, node-1, node-2
56	video-game-products	100 MB	node-0, node-1, node-2
57	video-game-products	100 MB	node-0, node-1, node-2
58	video-game-products	100 MB	node-0, node-1, node-2
59	video-game-products	100 MB	node-0, node-1, node-2
60	video-game-products	100 MB	node-0, node-1, node-2
61	video-game-products	100 MB	node-0, node-1, node-2
62	video-game-products	100 MB	node-0, node-1, node-2
63	video-game-products	100 MB	node-0, node-1, node-2
64	video-game-products	100 MB	node-0, node-1, node-2
65	video-game-products	100 MB	node-0, node-1, node-2
66	video-game-products	100 MB	node-0, node-1, node-2
67	video-game-products	100 MB	node-0, node-1, node-2
68	video-game-products	100 MB	node-0, node-1, node-2
69	video-game-products	100 MB	node-0, node-1, node-2
70	video-game-products	100 MB	node-0, node-1, node-2
71	video-game-products	100 MB	node-0, node-1, node-2
72	video-game-products	100 MB	node-0, node-1, node-2
73	video-game-products	100 MB	node-0, node-1, node-2
74	video-game-products	100 MB	node-0, node-1, node-2
75	video-game-products	100 MB	node-0, node-1, node-2
76	video-game-products	100 MB	node-0, node-1, node-2
77	video-game-products	100 MB	node-0, node-1, node-2
78	video-game-products	100 MB	node-0, node-1, node-2
79	video-game-products	100 MB	node-0, node-1, node-2
80	video-game-products	100 MB	node-0, node-1, node-2
81	video-game-products	100 MB	node-0, node-1, node-2
82	video-game-products	100 MB	node-0, node-1, node-2
83	video-game-products	100 MB	node-0, node-1, node-2
84	video-game-products	100 MB	node-0, node-1, node-2
85	video-game-products	100 MB	node-0, node-1, node-2
86	video-game-products	100 MB	node-0, node-1, node-2
87	video-game-products	100 MB	node-0, node-1, node-2
88	video-game-products	100 MB	node-0, node-1, node-2
89	video-game-products	100 MB	node-0, node-1, node-2
90	video-game-products	100 MB	node-0, node-1, node-2
91	video-game-products	100 MB	node-0, node-1, node-2
92	video-game-products	100 MB	node-0, node-1, node-2
93	video-game-products	100 MB	node-0, node-1, node-2
94	video-game-products	100 MB	node-0, node-1, node-2
95	video-game-products	100 MB	node-0, node-1, node-2
96	video-game-products	100 MB	node-0, node-1, node-2
97	video-game-products	100 MB	node-0, node-1, node-2
98	video-game-products	100 MB	node-0, node-1, node-2
99	video-game-products	100 MB	node-0, node-1, node-2

Figure 4: List of all shards

Our index is divided into three primary shards, each containing a portion of the dataset, and three replica shards, which store identical copies of their corresponding primaries. This results in six shards per index (e.g., *video-game-products* and *video-game-reviews*), distributed evenly across the nodes (see fig. 4; note that the shards containing dataset information are the only ones not prefixed by a period).

By partitioning the data across multiple shards, Elasticsearch can route each query directly to the shards that contain the relevant information, enabling parallel processing and faster search performance. This distribution also supports fault tolerance, as explained in Test 1, since replicas allow the system to continue functioning even if a node fails.

4.3 Test 3 — Indexing and Ingestion Performance

To evaluate ingestion performance, we used two Python scripts: *ingest.py* for the main review dataset and *ingest_file.py* for the meta-data. Each script reports its indexing summary upon completion.

The *ingest.py* script ingested 4,624,615 review documents over 37 minutes and 12 seconds, achieving an average throughput of approximately 2,072 documents per second (see fig. 5). This reflects the cost of indexing large text fields and performing distributed writes across three primary shards.

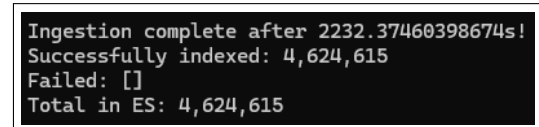


Figure 5: *ingest.py* ingestion time

The *ingest_file.py* script indexed 137,269 metadata documents in 1 minute and 32 seconds, corresponding to a rate of roughly 1,492 documents per second (see fig. 6).

```

Mapping created!
Starting bulk ingest...
Indexing...
Bulk ingest finished after 92.23126602172852s
✓ Indexed: 137269
X Failed: 0
Total in ES: 137,269

```

Figure 6: *ingest_file.py* ingestion time

These ingestion times largely depend on the structure of the dataset. Many entries contain substantial text fields—such as product descriptions and review content—that can reach thousands of characters. Additionally, some documents include long lists of categorical tags, increasing document size and indexing overhead. These characteristics naturally influence the total ingestion time.

4.4 Test 4 - Search Query Test

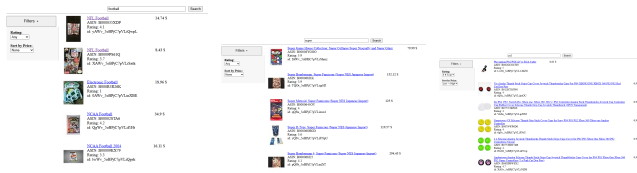


Figure 7: Examples for search queries

To evaluate distributed search, we executed several types of queries on our indices using the application interface (see fig. 7).

First, full-text match queries successfully returned all products whose titles contain the search term. These queries were processed in parallel across all three shards, and results were merged by the coordinating node.

Second, we tested aggregation queries, such as computing the average product rating. Each shard performed a partial aggregation on its local data, and Elasticsearch combined these partial results into a final value, demonstrating the distributed reduce phase.

Finally, the system supports filtered queries (e.g., filtering by rating or price). Filters are processed at the shard level, reducing unnecessary data transfer and improving performance.

Overall, these tests confirm that search operations—matching, filtering, and aggregations—are executed in a fully distributed and scalable manner across the cluster.

5 CONCLUSION

This project demonstrated the design and evaluation of a distributed data processing system built using a three-node Elasticsearch cluster and the Amazon Reviews dataset. Through our experiments, we showed how Elasticsearch’s distributed architecture enables the core properties of scalable systems: parallel ingestion, distributed search execution, and automatic shard balancing. By distributing 4.6 million documents across multiple primary and replica shards, the cluster was able to index and query large datasets efficiently while maintaining low response latency.

Our fault-tolerance tests confirmed that the system remains operational during node failure, with primary shards continuing to serve requests and replica shards ensuring data redundancy. Load balancing was observed across shards and nodes, allowing both ingestion and queries to run in parallel without bottlenecks. Overall, our results confirm that Elasticsearch effectively provides horizontal scalability, high availability, and resilient distributed search, meeting the key characteristics expected from a modern distributed system

REFERENCES

- [1] Elastic. 2025. *Discovery and Cluster Coordination*. <https://www.elastic.co/guide/en/elasticsearch/reference/current/modules-discovery.html> Accessed 2025-12-07.
- [2] Elastic. 2025. *Resilience in small clusters — Elasticsearch documentation*. <https://www.elastic.co/docs/deploy-manage/production-guidance/availability-and-resilience/resilience-in-small-clusters> accessed 2025-12-07.